ai-usage.md

Honest account of how I used AI (mostly Claude Code) to build this prototype.

What was AI-assisted

I have a pretty clear thesis on how to use AI. I use Claude for research synthesis, plan drafting, and code generation, but I don't let it make any final decisions or hide any ambiguities. I keep the human in the loop for all judgment calls, and I push back on any "just do this" suggestions that feel like safety theater or hand-waving.

Research phase (~1 hour): I asked an agent to fetch and synthesize ~25 Polymarket documentation pages in parallel, covering CLOB methods, Gamma response shapes, signature types, error codes, contract addresses, neg-risk markets, pUSD vs USDC, rate limits, and the order lifecycle. The same agent ran web searches across Stripe, CCXT, viem, Hyperliquid, the `polymarket-kit` prior art, and the facade-vs-adapter literature. The output of that phase is `docs/reference.md`.

Comprehension phase (~3 hours): I read everything the agent gathered, end to end, before letting it propose a design. Polymarket's docs first (every relevant page myself, not just the agent's summary), then the prior-art SDKs: the official `clob-client-v2` source on GitHub, `HuakunShen/polymarket-kit`'s structure, the `@dschz/polymarket-clob-client` community fork. I cross-referenced the agent's summaries against the source to catch places where it had paraphrased something subtly wrong (the `createOrDeriveApiKey` order-of-operations later turned out to be one of these). This is the phase where I formed my own opinion on what the right design should be: workflow-first surface, hide the fragmentation but don't deny the concepts, treat live trading as deserving real friction. By the end of this phase I knew what I wanted to build, not just what the agent was suggesting.

Planning phase (~1 hour): The agent surfaced three variants of the SDK shape (cheapest re-export, layered core+journeys, Stripe-style resource namespace). I made the call to go with the **Stripe-style resource namespace** because it's the shape that scales: `client.markets.get(...)`, `client.orders.place(...)`, `client.positions.list(...)` mirrors how Stripe organizes their SDK (`stripe.customers.create`, `stripe.charges.create`, `stripe.subscriptions.list`), maps 1:1 to Polymarket's REST organization (`/markets/`, `/orders/`, `/positions/`), and reads as the most defensible architectural choice for a reviewer who already knows what good SDK shapes look like.

The cheapest re-export option leaks `clob-client-v2` 's quirks into my surface; the layered core+journeys option adds structure for structure's sake. Stripe-style was the only option that earned its complexity.

The **facade + adapter** layering was something I worked out during planning, not something the agent proposed wholesale. Once I saw the SDK was going to wrap three different services, I realized I was combining two patterns from the classic GoF playbook (GoF = Gang of Four, the four authors of the foundational 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software*, which named most of the architectural patterns we still use today) at different layers: a **facade** at the top (`PolymarketClient` exposing a single coherent surface over Gamma + CLOB + Data API) and an **adapter** underneath (`ClobAdapter` wrapping `clob-client-v2` to conform to my internal type/error conventions). I asked the agent to confirm the GoF definitions, made sure the design rationale named these patterns explicitly, and forced the docs to be written in that vocabulary. Mixing the patterns without naming them is exactly what makes "wrapped SDK" submissions read as accidental rather than deliberate. A reviewer who recognizes "facade over services + adapter over the underlying SDK" has a clean mental model in two sentences.

I pushed back on the agent's first instincts on several other points where its default didn't match my judgment:

- **Real-money guardrail:** the agent initially recommended a single literal- `true` `confirm` gate. I argued the runtime env gate also pulls weight, because the two answer different questions: developer intent (call-site, compile-time) vs operational authorization (runtime, environment). A literal-only gate doesn't survive a hardcoded `confirm: true` in a script intended for staging running against a mainnet wallet. The agent dropped its "safety theater" framing and updated to a two-gate design (discriminated union + env gate).
- **Cred handling:** the agent's first version used `clob-client-v2` 's `createOrDeriveApiKey` directly. Once I hit the "Could not create api key" error and we traced the source, I made the call to invert to derive-first. The SDK's actual implementation tries create-first and the fallback path is dead code under our `throwOnError: true` setting, even though the official docs claim derive-first. We deviated from the official quickstart deliberately, and that's documented in design.md.
- **Diagnostic helper layout:** the agent offered three options (delete all / keep with npm scripts / move to `tools/` and wire as `diag:*`). I picked the `tools/` directory + npm scripts because it tells a real story about the integration debugging that actually happened, and stays discoverable for future readers without cluttering the repo root.

Output: `docs/plans/2026-04-29-001-feat-poly-unified-sdk-plan.md` .

Implementation phase (~5 hours): This was the longest phase by far. I worked through the eight implementation units (skeleton, types, errors, http + price math, GammaClient + normalize,

ClobAdapter + DataApiClient, facade, env loader + smoke + examples + docs) in dependency order. I wrote the inital code according to the design; I let AI assist; I made the architectural calls, reviewed every diff before moving on, ran `tsc --noEmit` and `vitest run` after each unit, and made the judgment calls when the agent surfaced ambiguity.

A lot of this phase wasn't really about writing code. It was about discovering things by running the SDK against real Polymarket APIs and feeding what I found back into the design:

- I wrote and ran the smoke test from a US connection, hit a Cloudflare 403 wall, and realized Polymarket geoblocks more aggressively than the docs let on.
- I traced the "Could not create api key" error end-to-end with the agent, which is how we ended up reading `clob-client-v2` 's source ourselves. I helped the agent discover that the official `createOrDeriveApiKey` quickstart pattern silently fails under `throwOnError: true` because the fallback path is unreachable. That discovery turned into the derive-first design choice and a dedicated section in design.md.
- I caught a real bug where `runWorkflow` was double-quoting (calling `calculateMarketPrice` twice, once in `quote()` and once again inside `place()` 's internal re-quote). The displayed quote could diverge from the actually-signed worst-price if the orderbook moved between the two calls. I directed the fix (pass the computed quote through into `place`) and asked for a regression test that uses `vi.fn().mockResolvedValueOnce(0.5).mockResolvedValueOnce(0.6)` so any future re-introduction of the bug fails loudly.
- I placed an actual $1 live trade through the SDK against a real Polymarket market, verified the on-chain transaction on Polygonscan, and confirmed the position read-back through Data API. This is the strongest piece of integration evidence the submission has.
- I directed the place-order example to print richer details (market URL, funder Polygonscan URL, full transaction URLs, the quote that drove the order, position PnL with proper +/- signs) and a 5-second wait before live submission instead of 3.
- One real bug got caught by a fuzz test that the agent had written: the SELL clamp interaction with the `tickSize` floor. The agent surfaced this as a semantic question rather than papering over it, which let us narrow the test invariant rather than silently weaken it.

Total tests at the end: 153, all passing, zero network calls in the suite.

What was NOT AI-driven

- **Final design choices:** Every load-bearing decision (resource-oriented namespace, two-gate guardrail, dryRun as a discriminated-union mode, in-memory cred caching, Polygon-only scope)

was a judgment call. The agent presented options and tradeoffs and I made the design decisions.

- **The "best of both" synthesis on the live-trading gate:** The agent initially argued the env gate was "safety theater." I disagreed; env and call-site gates answer different questions (operational authorization vs developer intent). The agent updated its position after the comparison.
- **The choice to absorb the existing `PolymarketWorkflowClient` patterns** (DI seams, tolerant Gamma normalization, named error taxonomy, `validateFokOrderResponse`, tick rounding direction, env loader with group rules). I directed which patterns to keep and which to refine.

What I would change if doing this without an agent

Probably not much in the design. The design judgment is mine; the agent just produced rationale tables and parallel research faster than I could.

The implementation would be slower without the agent, though. The DI seam pattern would still be there, the tolerant Gamma normalization would still be there, the discriminated union for the guardrail would still be there. They'd just take longer to write, and the test coverage would probably be thinner because manual test authoring is more tedious than directing an agent to enumerate scenarios from a spec.

Specific agent interactions worth flagging

1. **Surfacing prior art:** The agent found `HuakunShen/polymarket-kit` early in the research phase, recognized it as direct prior art, and used the comparison to justify the orchestration-layer design choice. Without that, I might have started from scratch and missed the "the gap they didn't fill" framing.
2. **Catching the tick-rounding semantic edge:** The fuzz test for SELL rounding produced a real failure: when SELL slippage pushes the worst-price below `tickSize`, the clamp restores it to `tickSize`, which is *better* for the seller, but breaks the "rounded ≤ unrounded" invariant in that region. The agent surfaced this as a question rather than silently fixing it. We narrowed the test to non-clamped territory and documented the behavior.
3. **Help in writing tests for edge cases:** The agent enumerated a wide range of edge cases for the FOK quote and place workflows, which led to a much more robust test suite than I would have written on my own. For example, the

`vi.fn().mockResolvedValueOnce(0.5).mockResolvedValueOnce(0.6)` pattern for catching the double-quote bug.

Tooling

- **Claude Code + 4 personal, custom agents** for research synthesis, plan drafting, code generation, and code review.
- Standard development tools: `tsc`, `vitest`, `npm`, `tsx`. No codegen tooling. Vim for coding.